

Intro to Linux Commands

BASIC CONCEPTS

A **terminal** is a "physical"(direct) interface to your Linux Operating System.

A **shell** is an interpreter for commands entered into the terminal. It is used to call commands and programs

- Many available (bsh; csh; **bash**; etc.)
- The **Bourne Again Shell (bash)** is a popular shell on Linux systems.

WHY USING A TERMINAL?

1. It can save you time.
2. It can help when you are unable to use the GUI, such as a system crash or a configuration issue.
3. It can enable you to use Linux in ways that using a GUI exclusively can not (such as scripting repetitive tasks).

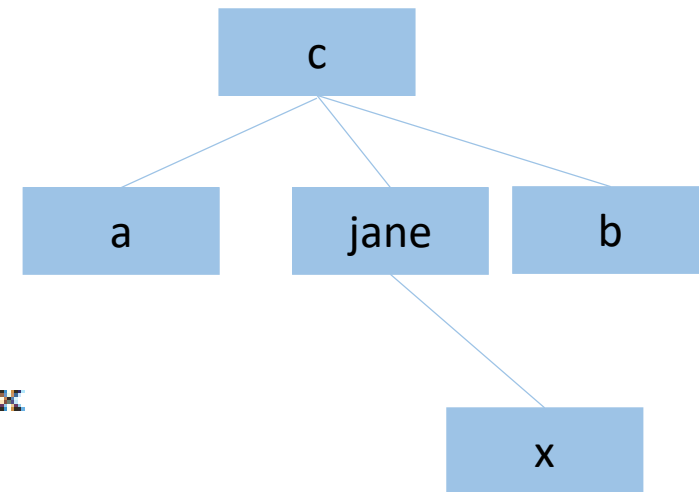
SPECIAL FILE NAMES

- Some file names are special:

- `/` The root directory (not to be confused with the root user)
- `.` The current directory
- `..` The parent (previous) directory
- `~` My home directory

- Examples:

- `./a` same as `a`
- `../jane/x` go up one level then look in directory `jane` for `x`



HELP IN LINUX

The Linux equivalent of HELP is man
(manual)

- Use `man -k <keyword>` to find all commands with that keyword
- Use `man <command>` to display help for that command
 - Output is presented a page at a time. Use `b` for to scroll backward, `f` or a space to scroll forward and `q` to quit

NAVIGATION AND LOOKING AROUND

- `pwd` - print (display) the working directory
- `cd <dir>` - change the current working directory to *dir*

```
cd ..
```

- `ls` - list the files in the current working directory
- `ls -l` - list the files in the current working directory in long format
- `file` show info about the file (type, date of creation)

FILE AND DIRECTORY MANIPULATION

- `cp <fromfile> <tofile>`
 - Copy from the <fromfile> to the <tofile>
- `mv <fromfile> <tofile>`
 - Move/rename the <fromfile> to the <tofile>
- `rm <file>`
 - Remove the file named <file>
- `mkdir <newdir>`
 - Make a new directory called <newdir>
- `rmdir <dir>`
 - Remove an (empty) directory

File Permissions

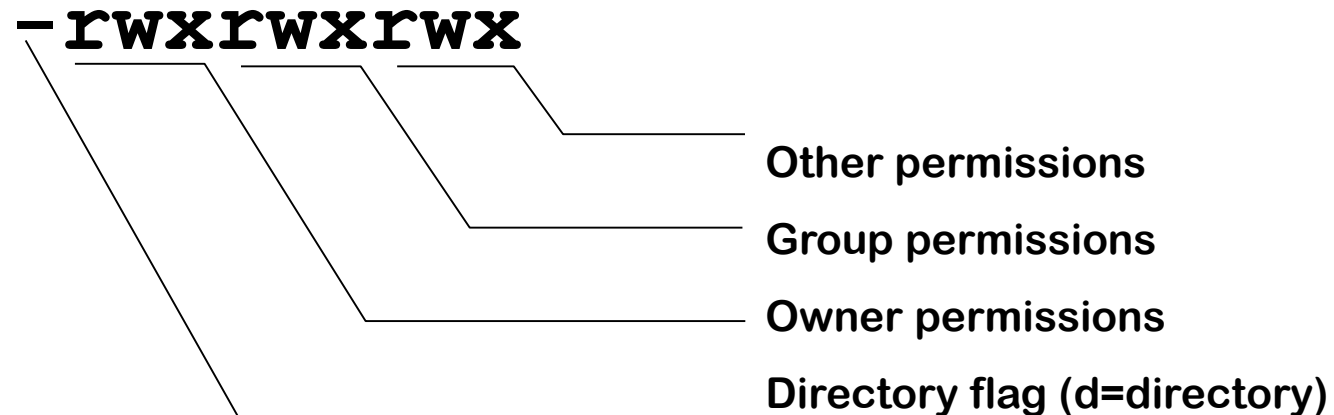
Each file and folder has certain permissions assigned. They define who is authorized to write, read or execute this file.

There are three types of access:

- **read**: the file can be only read
- **write**: the file can be edited
- **execute**: the file can be executed as a program

There are three types of user groups, these access types can be applied to:

- **owner**: the owner of the file
- **group**: other files which are in the same folder or group
- **world**: anyone else



File Permissions

- The access level is defined in numbers:

0 - no access to the file

1 - execute only

2 - write only

3 - write and execute

4 - read only

5 - read and execute

6 - read and write

7 - read, write and execute (full permissions)

0	none	---
1	execute-only	--x
2	write	-w-
3	execute and write	-wx
4	read-only	r--
5	read and execute	rx-
6	read and write	rw-
7	read, write, and execute	rxw

Setting Permissions

- When you wish to set the mode of a file (set the permissions) you use the UNIX command **chmod** at the system prompt. Try using the **-v** option for a verbose response as in the following example:
- ```
chmod -v 640 myfile.txt
```
- ```
mode of `myfile.txt' changed to 0640 (rw-r-----)
```
- This command designates that the file named **myfile.txt** has read and write (rw-) permission for the owner (you), read-only (r-) permission for the group members, and no access permissions for others (—).
- Remember that the permissions for “owner” are always first and the permissions for “other” are always last.
- You can also use the letters r, w, and x to set read, write, and execute permissions and the letters u, g and a to specify user, group or all:
- ```
chmod a+w myfile.txt
```
- The above adds the write permission for all users.

# Setting Permissions

| Command              | Permissions |
|----------------------|-------------|
| chmod 777 myfile.txt | rw-rw-rw-   |
| chmod 720 myfile.txt | -rwx-w---   |
| chmod 744 myfile.txt | -rwxr--r-   |
| chmod 754 myfile.txt | -rwxr-xr-   |
| chmod 750 myfile.txt | -rwxr-x---  |

## More Commands

**grep**: Searches files for one or more pattern arguments. It does plain string, basic regular expression, and extended regular expression searching

**wc**: It displays four-columnar output. First column shows number of lines present in a file specified, second column shows number of words present in the file, third column shows number of characters present in file and fourth column itself is the file name which are given as argument. (options -l , -w, -m)

**sort**: options -r

**find** : Searches a given file hierarchy specified by path, finding files that match the criteria given by expression

## Standard I/O Streams

Standard I/O streams are pre-connected input and output communication channels between a computer program and its environment when it begins execution.

| Standard stream | Purpose                                                                                                            | Default endpoint |
|-----------------|--------------------------------------------------------------------------------------------------------------------|------------------|
| <b>stdin</b>    | <b>Standard input</b> is the stream of data data (often text) going into a program.                                | <b>terminal</b>  |
| <b>stdout</b>   | <b>Standard output</b> is the stream where a program writes its output data.                                       | <b>terminal</b>  |
| <b>stderr</b>   | <b>Standard error</b> is another output stream typically used by programs to output error messages or diagnostics. | <b>terminal</b>  |

# File Descriptors

File descriptor values for the standard I/O streams

| File descriptor value | Standard stream | Mode  | Endpoint |
|-----------------------|-----------------|-------|----------|
| 0                     | stdin           | read  | terminal |
| 1                     | stdout          | write | terminal |
| 2                     | stderr          | write | terminal |



## Redirection into a file

- Each stream uses redirection commands. Single bracket '>' or double bracket '>>' can be used to redirect standard output.
- If the target file doesn't exist, a new file with the same name will be created.

### Redirect to stdout

- `ls -l > file.txt`
- `ls -l >> file.txt`
- `ls -al file.txt > file2.txt`
- `cat file.txt file2.txt > file3.txt`    #combines files

### Redirect to stderr

- `ls -al sdghksdg.txt 2> file2.txt`
- Try `2>>` as well

# Input Redirection

- The bash shell uses stdin to take input.
- In input redirection, a file is made input to the command and this redirection is done with the help of '<' sign.
- Eg. *cat < file.txt*

## Combined Example:

- Let's say we have a "words.txt" file containing "car,airplane,train"
- `sort words.txt`
- `sort < words.txt > words2.txt`

# Pipes

- The Pipe is a command in Linux that lets you use two or more commands such that output of one command serves as *input* to the next.
- When a program's standard output is sent to another through a pipe, the first program's data, which is received by the second program, will not be displayed on the terminal.
- Only the filtered data returned by the second program will be displayed.
- The Linux pipe is represented by a vertical bar \* | \*

## Examples:

- `cat words | sort`
- `cat words | sort -r`
- `cat words | grep e`
- `ls -l | grep Feb`
- `cat words | grep e > file`

# Multiple Commands

**To execute 2 commands consecutively:**

```
command1 ; command2
```

**To execute command 1 successfully before executing command2:**

```
command1 && command2
```

**To execute command2 only if command1 fails:**

```
command1 || command2
```